

Andrei Nicolae BESLEAGA¹

¹Affiliation not available

February 27, 2026

GABBE: A Neurocognitive Swarm Architecture for Agentic AI Software Engineering

Authors: BESLEAGA Andrei Nicolae, Independent Researcher

Corresponding Author: andrei.besleaga.nicolae@gmail.com

Abstract The transition from human-centric to agent-centric software engineering has revealed critical limitations in existing Multi-Agent Systems (MAS), particularly concerning context decay, non-determinism, and cost asymmetry. This paper introduces the Generative Architectural Brain Base Engine (GABBE), a third-generation autonomous software system built upon neurocognitive principles. GABBE features a dual-layer architecture: a meta-cognitive “Brain Mode” utilizing Active Inference to minimize project risk, and a “Loki Swarm Mode” executing deterministic, spec-driven development life cycles across 30+ specialized agent personas. This paper details the system’s 4-layer memory hierarchy, cost-effective Large Language Model (LLM) routing, and the programmatic enforcement of the “Another Lethal Trifecta” mitigation strategy. A comparative analysis demonstrates GABBE’s architectural superiority over flat-topology orchestrators, providing a scalable blueprint for applying neurocognitive agentic patterns to zero-to-one software development.

Index Terms Agentic AI, Swarm Intelligence, Active Inference, Multi-Agent Software Engineering, Cognitive Architecture, Large Language Models, Software Architecture, Neuroscience, Software Engineering.

I. INTRODUCTION

The industrialization of Agentic AI software engineering has moved beyond the stochastic “Vibe Coding” era of 2024-2025, which, while democratizing code generation, resulted in unmaintainable codebases and high hallucination rates. The maturation of the field necessitates treating AI not as a probabilistic oracle, but as a deterministic component within a highly constrained engineering system. Early attempts at autonomous engineering encountered what is now termed “Another Lethal Trifecta”—three compounding failure modes identified by industry analysts:

1. **Velocity Mismatch:** AI agents generate code orders of magnitude faster than human reviewers can validate it. Without automated architectural constraints, technical debt accumulates at machine speed, rendering codebases immutable due to complexity.
2. **Non-Determinism:** Unlike a compiler, an agent given the same input twice may produce divergent outputs. Standard prompt-based coding fails because it relies on probabilistic “luck” rather than a repeatable engineering process.
3. **Cost Asymmetry:** While individual tokens are inexpensive, the “context rot” associated with poorly managed agent sessions—where an agent spirals into a loop of reading and re-reading irrelevant files—causes operational costs to balloon. Efficient agents require constrained context.

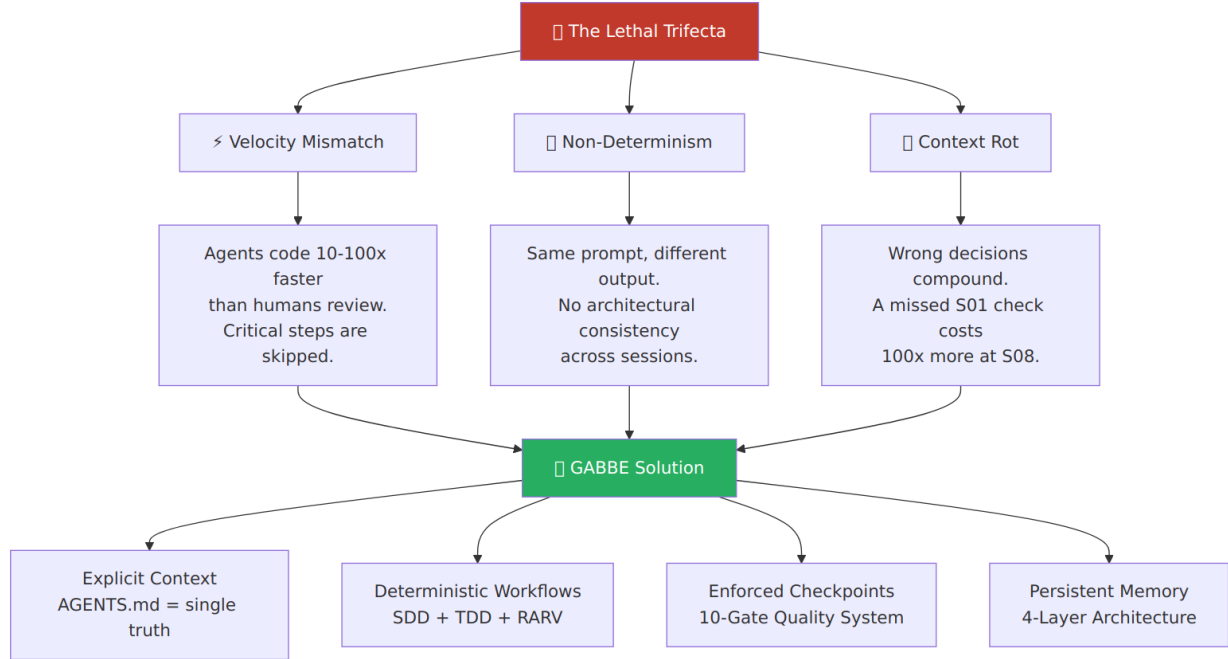


Fig 1. The compounding failure modes of unmanaged Agentic AI.

To overcome these barriers, the Generative Architectural Brain Base Engine (GABBE) proposes a paradigm shift. GABBE is not merely a framework but a pre-configured “Engineering Organization in a Box.” It implements a dual-layer cognitive architecture inspired by Daniel Kahneman’s System 1 and System 2 processing, mapping operational execution to a swarm of specialized agents (Loki Mode) and strategic orchestration to a meta-cognitive planner (Brain Mode). This paper details the architectural innovations of GABBE, validating its design against recent literature in Active Inference and Swarm Intelligence [1], and demonstrating how these principles can be applied to architect future AI agentic engineering systems from zero.

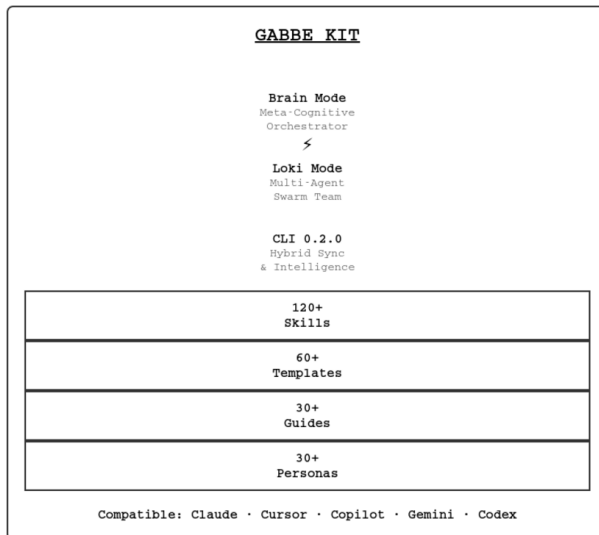


Table I. GABBE’s paradigm as a Universal Engineering Context Layer relative to existing AI models.

II. BACKGROUND AND RELATED WORK

The convergence of autonomous agents and software engineering relies heavily on advancements in three domains: Swarm Intelligence, Active Inference, and LLM-Based Multi-Agent Software Engineering.

A. Agentic AI and Multi-Agent Software Engineering

Recent surveys (e.g., “Agentic AI: Autonomous Intelligence for Complex Goals,” 2025) categorize Agentic AI by autonomy, efficacy, and goal complexity [8]. In software engineering, frameworks like ALMAS (Autonomous LLM-based Multi-Agent Software Engineering Framework, 2025) have proposed aligning agents with Agile roles to automate the Software Development Life Cycle (SDLC) [2]. While frameworks like CrewAI and LangGraph excel at defining execution graphs and role-based assignments, they inherently lack domain-specific software engineering intelligence and long-horizon risk management.

B. Swarm Intelligence

Swarm intelligence algorithms have evolved from simple particle optimization to collaborative LLM model swarms (e.g., “Model Swarms: Collaborative Search to Adapt LLM Experts via Swarm Intelligence,” 2024) [1]. Decentralized decision-making capabilities within swarms are increasingly integrated with deep reinforcement learning environments to manage complex, dynamic interactions among agents.

C. Active Inference in AI

Derived from the Free Energy Principle (FEP) pioneered by Karl Friston, Active Inference posits that agents act to minimize variational free energy—the difference or “surprise” between an internal generative model and actual sensory inputs [3]. Recent papers (e.g., B. Wen, “The Missing Reward: Active Inference in the Era of Experience,” 2025) advocate replacing external reward signals with intrinsic surprise minimization, enabling agents to learn from experience without continuous human reward engineering [4].

III. THE GABBE COGNITIVE ARCHITECTURE

GABBE’s core innovation lies in its treatment of the software system as a Cognitive Entity rather than a state machine. It introduces a structural separation spanning procedural, semantic, and episodic dimensions.

A. Dual-Layer Orchestration (Brain and Loki Modes)

GABBE’s core capabilities map directly to the pain points of unmanaged AI development, providing specialized mechanisms for governance, memory, and task resolution.

Layer	What	Count
Skills	Specialized agent capabilities (SOPs)	120+
Templates	Fill-in-the-blank engineering documents	60+
Guides	Language & domain expertise textbooks	30+
Personas	Specialized agent roles	30+
Brain Skills	Neuro-cognitive orchestration modules	10+
SDLC Gates	Mandatory quality checkpoints	10

Table II. Summary of GABBE’s primary governance and operational capabilities.

GABBE orchestrates tasks through a hierarchical topology, visually represented in the hybrid state synchronization model:

- **Brain Mode (System 2 - The Strategist):** A meta-cognitive orchestrator that functions above the SDLC. Inspired by Active Inference, it continuously predicts project outcomes, observes actual codebase states, and adapts organizational strategies to minimize “surprise” (i.e., bugs, security risks, cost overruns).

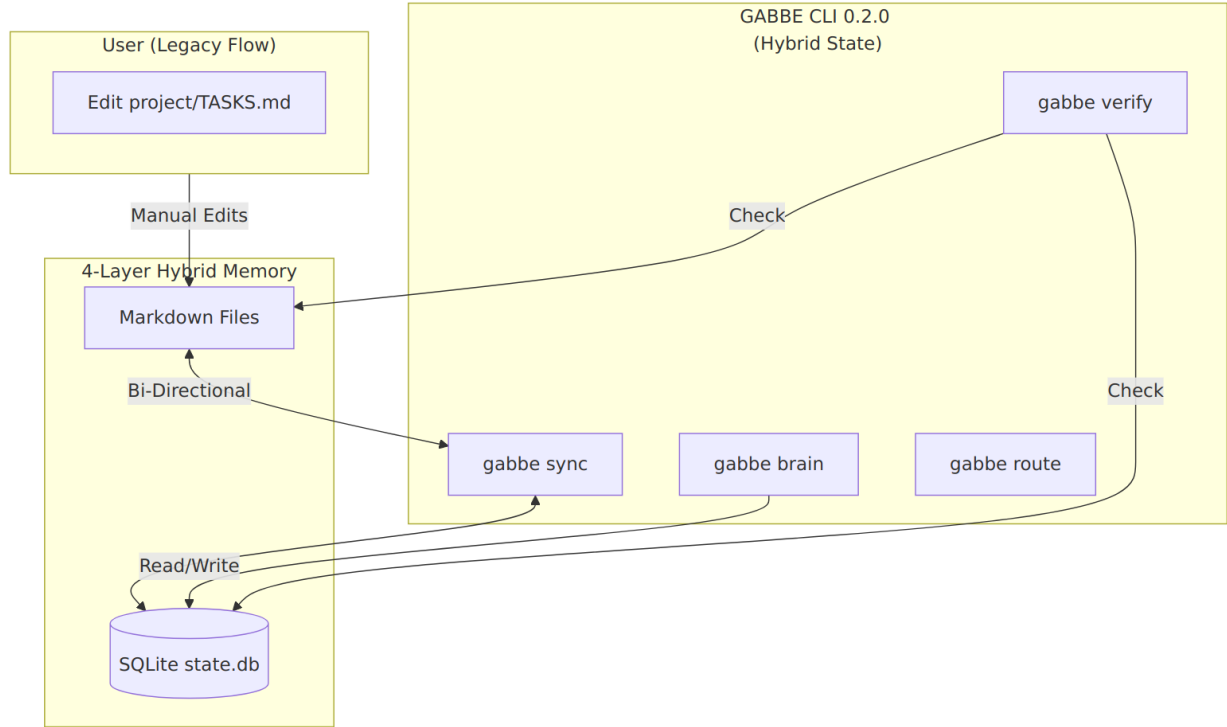


Figure 1: Hybrid State Synchronization Model

- **Loki Mode (System 1 - The Executor):** A multi-agent swarm comprising 30+ specialized personas divided into functional sub-swarms: Engineering (The Builders), Operations (The Shield), Product (The Visionaries), and Orchestration (The Managers). Loki executes deterministic SDLC governance (Phases S0 to S10).

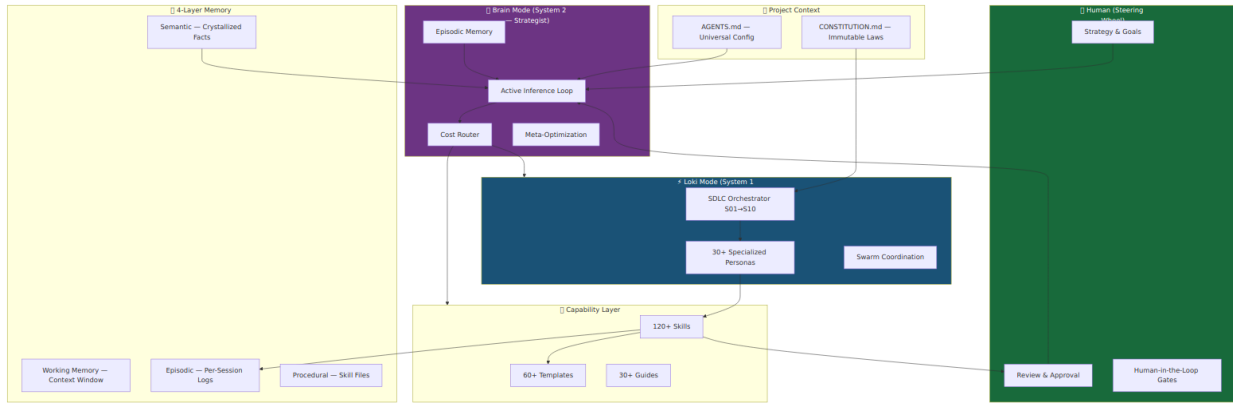


Fig 2. The high-level hierarchy of the GABBE platform.

To enable the Brain Mode's Active Inference functionality, GABBE implements a dedicated subset of meta-cognitive skills representing human project management duties:

Cognitive Function	Skill	Analogy
Coordination	<code>global-workspace.skill.md</code>	The brain's "spotlight" of attention
Control Loop	<code>active-inference.skill.md</code>	Predictive coding — minimize surprise
Self-Awareness	<code>consciousness-loop.skill.md</code>	Recursive OODA + Strange Loop
Embodied Action	<code>sensory-motor.skill.md</code>	Tools as muscles with proprioception
Evolution	<code>self-improvement.skill.md</code>	Rewriting its own "DNA" (prompts)
Session State	<code>working-memory.skill.md</code>	Short-term project context
Long-term Storage	<code>episodic-consolidation.skill.md</code>	Vector-stored decisions
Cost Intelligence	<code>cost-benefit-router.skill.md</code>	Dynamic local vs. remote switching
Theory Base	<code>neuroscience-foundations.skill.md</code>	Cortico-Thalamic & Basal Ganglia

Table III. Specialized skills utilized by the System 2 meta-cognitive orchestrator.

B. Meta-Cognition and Global Workspace Theory

GABBE transcends standard System 1/System 2 divisions by introducing explicit self-monitoring mechanisms. This meta-cognitive capability—often conceptualized as “thinking about thinking”—allows the Brain Mode to dynamically govern **Strategy Selection** (e.g., choosing TDD versus Spike-First approaches), **Resource Allocation** (routing complex tasks to state-of-the-art models), **Failure Detection** (circuit breakers for looping agents), and **Skill Gap Detection**.

To disseminate these high-level realizations across the swarm, GABBE implements a computational reflection of **Global Workspace Theory (GWT)**. Insights accumulated in the Working Memory are systematically broadcast to all localized Loki agents, simulating the cognitive “spotlight of attention” and ensuring the multithreaded multi-agent ecosystem globally aligns with shifted strategic objectives without causing individual context saturation.

C. The RARV Cognitive Loop

To constrain the swarm mathematically during the execution of these strategies, GABBE implements the RARV (Reason-Act-Reflect-Verify) cycle: 1. **Reason:** Cross-reference task definitions (`project/tasks.md`) with continuous context (`AGENTS.md`, `CONTINUITY.md`). 2. **Act:** Execute code modifications or infrastructure updates via the Model Context Protocol (MCP). 3. **Reflect:** Perform self-critique on the deterministic output prior to completion. 4. **Verify:** Programmatically validate code via automated linters, type checks, and TDD mechanisms.

D. The 4-Layer Memory Hierarchy

Standard LLM sessions suffer from “amnesia.” GABBE preserves context through an integrated structural memory approach:

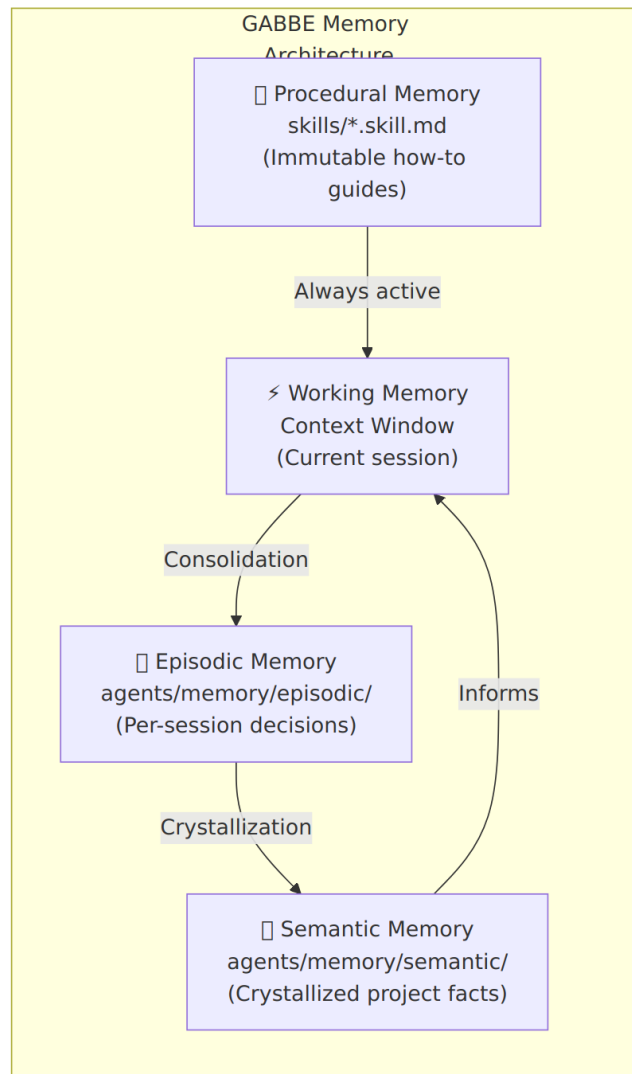
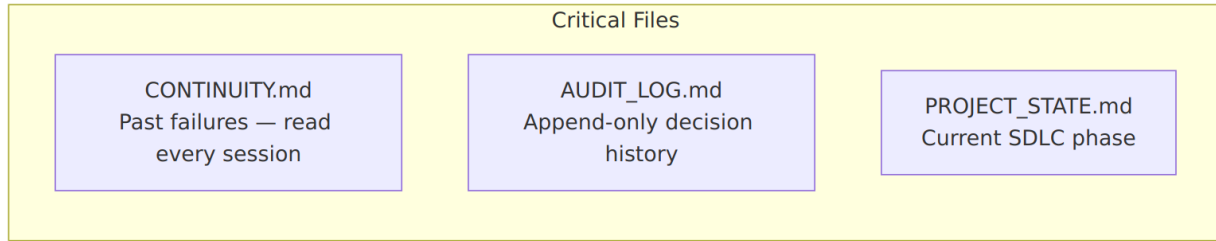


Fig 3. The separation of EPS (Episodic, Procedural, Semantic) memory layers. 1. **Working Memory:** The immediate context window (e.g., current task and file). 2. **Episodic Memory (memory/episodic/):** Session snapshots and decision logs, preventing the agent from repeating failed code experiments. 3. **Semantic Memory (memory/semantic/):** Crystallized, factual project knowledge (e.g., specific JWT authentication patterns required). 4. **Procedural Memory (skills/):** 130+ immutable .skill.md files acting as “how-to” guidelines.

E. The Agentic Stack (MCP and AGENTS.md)

GABBE standardizes environmental interactions through a modular separation of concerns, metaphorically structured as “Brains” (internal procedural memory) and “Hands” (external interfaces).

The Hands Layer (MCP): The Model Context Protocol (MCP) acts as the secure boundary, exposing local databases (e.g., PostgreSQL introspection) and vector storage mechanisms without exposing the core agentic loop to arbitrary code execution risks. By configuring an `mcp_config.json`, organizations explicitly whitelist external systems.

The Brain Layer (AGENTS.md and SKILLS.md): The `AGENTS.md` file serves as the universal “Single Source of Truth” configuration, symlinked across various AI providers to ensure all agents operate under identical architectural constraints and testing paradigms. Furthermore, procedural knowledge is managed via **Progressive Disclosure** in `SKILLS.md` files: the agent is initially aware only of a skill’s existence via a lightweight index. The full instruction set and executable logic are loaded into the context window *only* when the agent deliberately invokes that specific skill, mitigating context rot.

F. Meta-Evolution and Self-Healing

A pivotal feature of GABBE is its capacity for self-evolution (`meta-optimize.skill`). When the orchestrator detects recurring failures (e.g., an agent failing to implement a specific authentication library thrice), the system analyzes the root cause and autonomously proposes patches to its own `.skill.md` files. This establishes a continuous improvement loop where procedural memory hardens over the lifecycle of the project.

IV. IMPLEMENTATION DETAILS AND STRUCTURAL BLUEPRINTS

GABBE’s logic is grounded in a Hybrid Architecture that bridges human-readable Markdown files with a zero-dependency CLI written in Python, utilizing an embedded SQLite state abstraction [5][6].

A. Hybrid State and Synchronization

The `gabbe sync` protocol bidirectionally synchronizes human-edited `TASKS.md` with a SQLite database (`state.db`), guaranteeing that both the human architect and the AI swarm operate on a unified truth.

```
# Extracted snippet: gabbe/brain.py - Active Inference Prediction Engine
def activate_brain():
    # 1. Observation
    c.execute("SELECT status, count(*) FROM tasks GROUP BY status")
    stats = dict(c.fetchall())
    state_desc = f"Project State: {stats.get('TODO')} TODO..."

    # 2. Prediction & Action Selection via LLM
    prompt = f"Current Reality: {state_desc}\nPredict likely outcome and select minimizing action."
    action = call_llm(prompt, system_prompt="You are the Meta-Cognitive Brain...")
```

B. Cost-Benefit LLM Routing

Mitigating the “Cost Asymmetry” vector of the Lethal Trifecta, GABBE introduces a deterministic LLM cost router (`gabbe route`). It estimates prompt complexity via heuristics and secondary LLM queries.

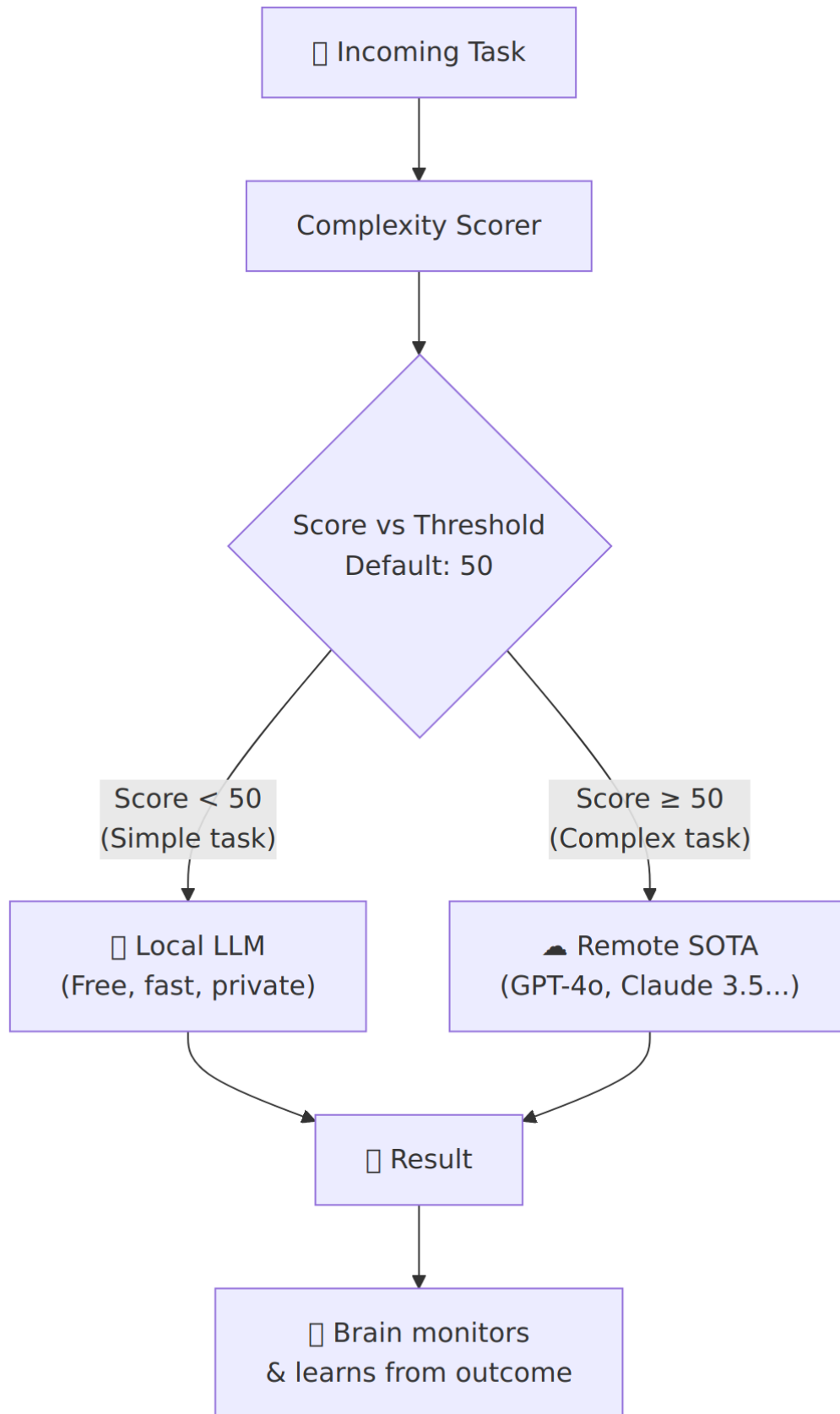


Fig 4. Decision

flow for local vs remote LLM task resolution.

```
# Extracted snippet: gabbe/route.py - Evolutionary Cost Arbitration
def route_request(prompt):
    if detect_pii(prompt):
        return "LOCAL" # Force secure local execution
    complexity, _ = calculate_complexity(prompt)
    return "REMOTE" if complexity > ROUTE_COMPLEXITY_THRESHOLD else "LOCAL"
```

Drawing from recent industry approaches optimized in frameworks like *RouteLLM* [7] involving preference-based routing classifiers, this algorithm should achieve 30-70% reductions in token expenditures by executing boilerplate UI tasks or privacy-sensitive data parsing on localized hardware, reserving state-of-the-art inference for architectural synthesis.

V. DETAILED SYSTEM ARCHITECTURE AND DATA FLOWS

To fully grasp the mechanism by which GABBE transforms isolated agents into a governed engineering team, it is essential to explore the functional anatomy of its architecture, the explicit stages of its development lifecycle, and the end-to-end data flows.

A. The Cognitive Turing Machine

GABBE is modeled as a cognitive entity, dividing responsibilities among anatomical equivalents: * **Prefrontal Cortex (Brain Mode)**: Strategic planning, cost routing, and Active Inference. * **Hippocampus (Memory Systems)**: The 4-Layer Hybrid Memory (Working, Episodic, Semantic, Procedural) preventing context rot and repeating failures. * **Motor Cortex (Loki Swarm)**: Executing the deterministic SDLC through 30+ specialized personas. * **Sensory Input (AGENTS.md & CLI)**: The environmental perception constraints governing tech stack and rules.

This structure allows the system to not just execute code, but to *reflect* on its execution against organizational goals.

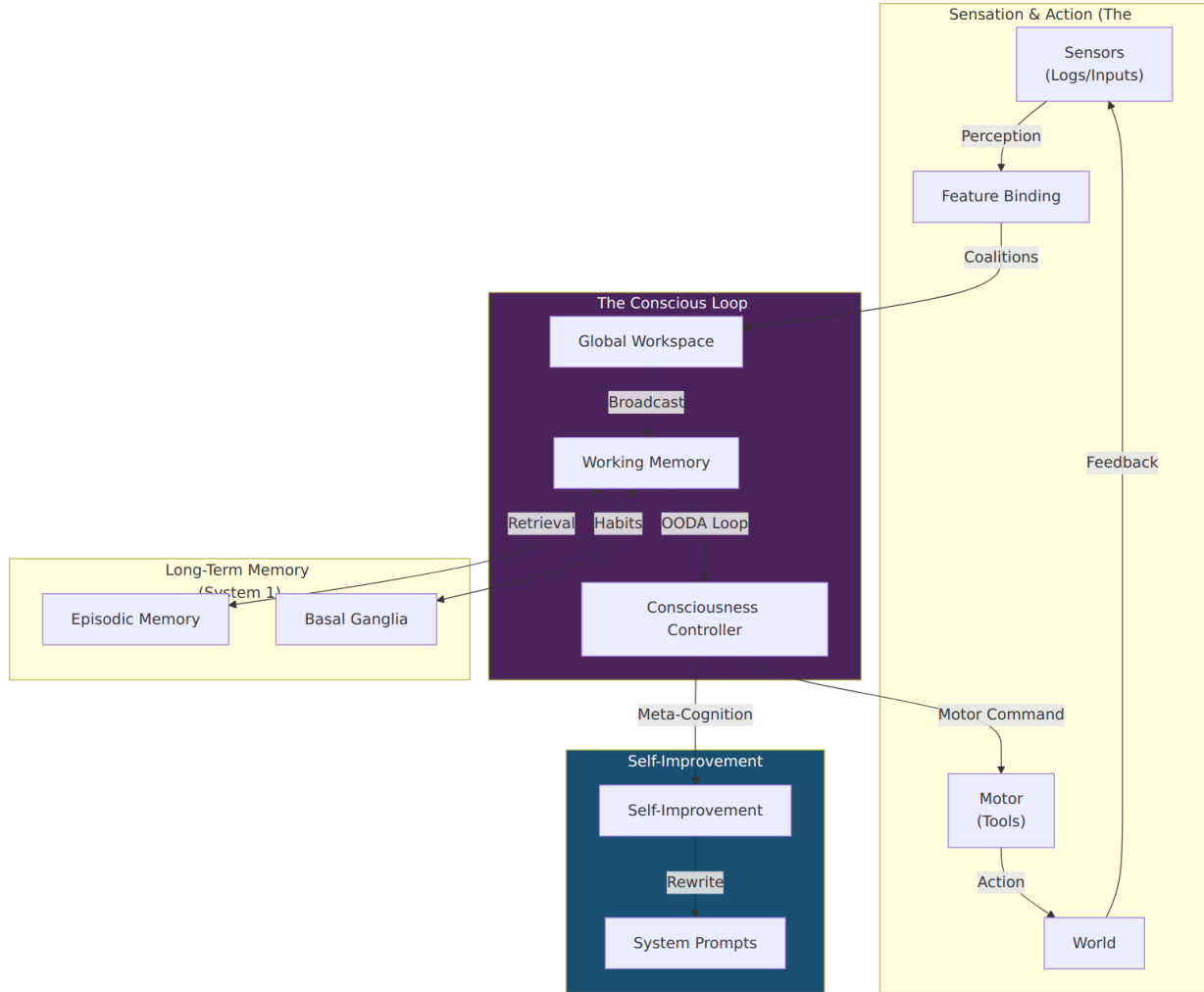


Fig 5. Cognitive Architecture Anatomy mapping to organizational layers.

B. The 10-Gate Quality System (Spec-Driven SDLC)

A core mitigation strategy against Velocity Mismatch is GABBE's strict Spec-Driven Software Development Life Cycle (SDLC). Beginning with a preliminary Strategy phase (S0), the swarm must pass through 10 sequential gates (S01–S10), each requiring automated heuristic checks and, at critical junctures, explicit human approval. The phases are:

- **S0 Strategy:** Validating business requirements and objectives.
- 1. **S01 Requirements:** Crafting the Product Requirements Document (PRD) using EARS syntax.
- 2. **S02 Design:** Defining architectural Context, Containers, and Components (C4 model) and ADRs.
- 3. **S03 Specification:** Specifying API contracts and data schemas.
- 4. **S04 Tasks:** Decomposing specifications into atomic, 15-minute executable tasks.
- 5. **S05 Implementation:** The RARV Loop execution governed by Test-Driven Development (TDD).
- 6. **S06 Testing:** Comprehensive unit, integration, and E2E testing.
- 7. **S07 Security:** Mandatory vulnerability scanning and RBAC verification.
- 8. **S08 Code Review:** Automated agentic code review against CONSTITUTION.md and human approval.
- 9. **S09 Staging:** Validating smoke tests and benchmarks in a pre-production environment.
- 10. **S10 Production:** Monitoring, logging setup, and final project hand-off.

Within **S01 Requirements**, GABBE mandates **Spec-Driven Development (SDD)** utilizing the **EARS**

(**Easy Approach to Requirements Syntax**) grammar. Because agentic reasoning strictly struggles with ambiguity, natural language PRDs are converted into deterministic state-machine templates: * **Ubiquitous:** “The System shall [Response].” (Global constraints) * **Event-Driven:** “When [Trigger], the System shall [Response].” (Reactive logic) * **State-Driven:** “While [State], the System shall [Response].” (Conditional logic) * **Unwanted:** “If [Condition], the System shall [Response].” (Error handling) * **Complex:** “Where [Feature], the System shall [Response].” (Variant behavior)

By translating human intent into EARS before writing any code, the system eliminates probabilism from feature acceptance.

C. End-to-End Workflow Data Flow

The orchestration between the human user, Brain Mode, Loki Mode, and the storage layer operates sequentially to enforce governance:

This rigorous data flow ensures that every line of code generated is explicitly tied back to an approved semantic requirement (PRD), and verified through procedural limits (Skills), drastically reducing the hallucination surface area common in unbound Agentic AI.

D. The Loki Swarm Architecture

To fulfill operations across all SDLC gates, the underlying swarm operates via highly contextualized personas mapping directly to conventional engineering department roles. This separation of “Planner” and “Executor” functionally mirrors recent academic topologies, such as those proposed in *EvoMAC* (ICLR 2025) [9], specifically isolating the “Judge” agent pattern (**orch-judge**) to ensure unbiased quality gates independent of the code generation swarm.

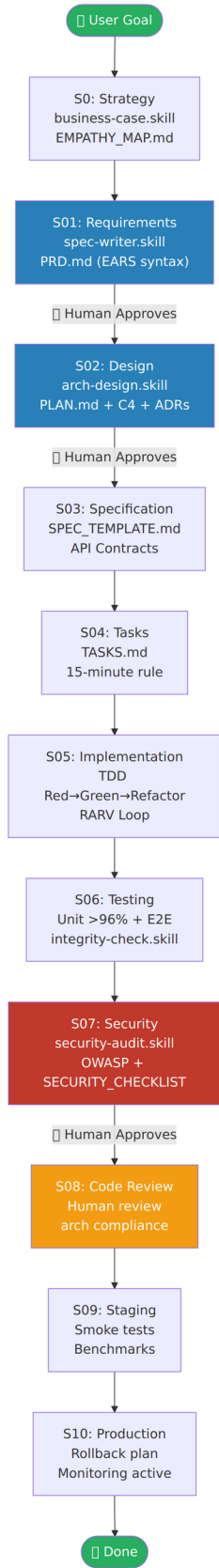
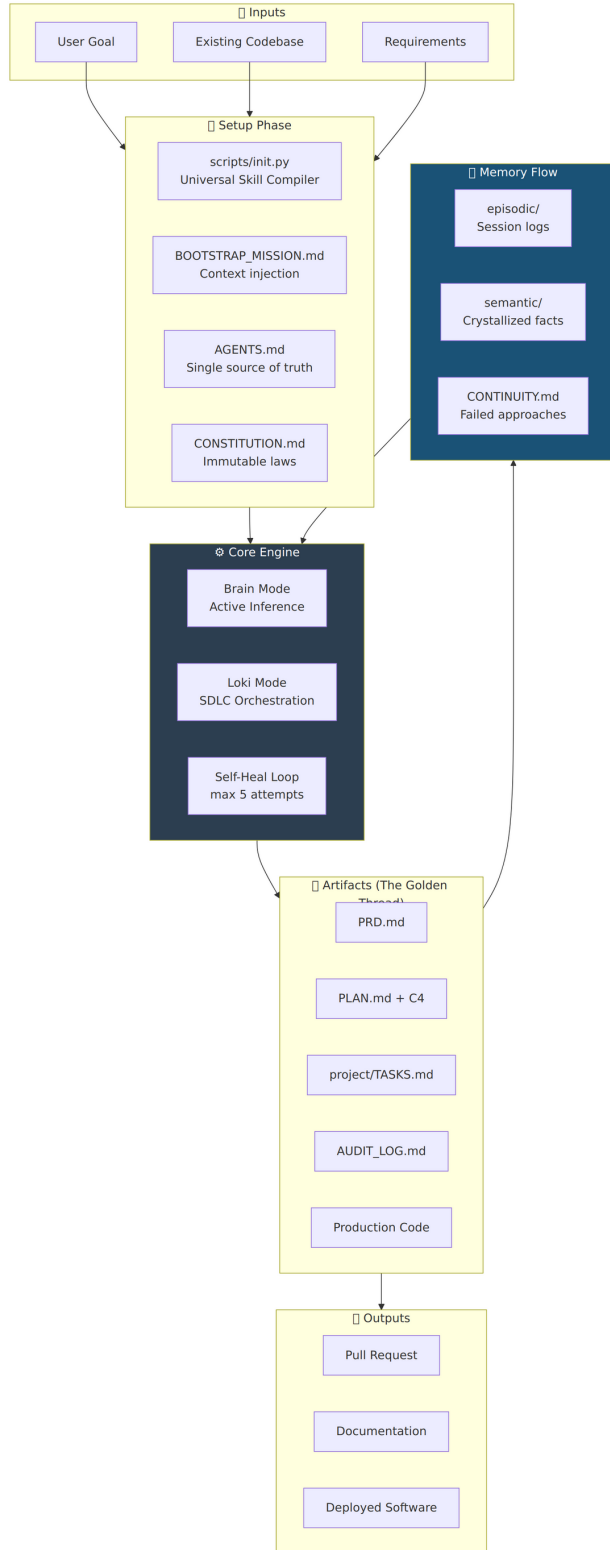


Fig 6. The 10-Gate Spec-Driven Software Development Life Cycle (SDLC).



manual edits to the hybrid core.

Fig 7. End-to-End Workflow Data Flow connecting

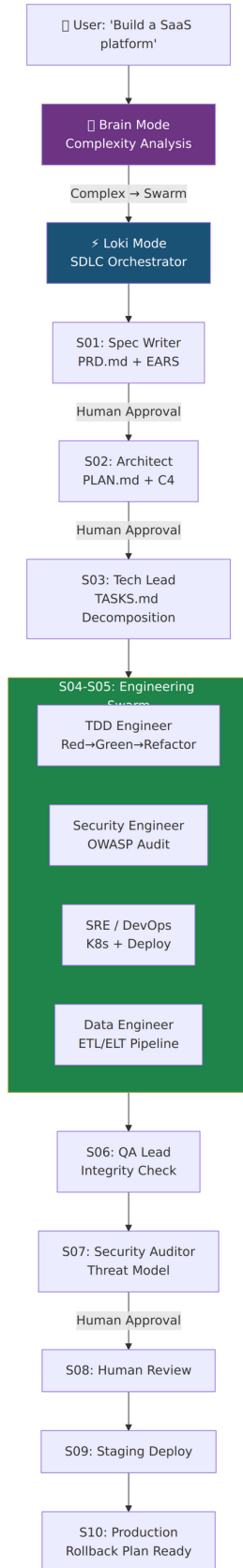


Fig 8. Functional groupings and context transition within Loki Mode.

E. Architecture Patterns and Engineering Best Practices

To maximize the efficiency of the swarm, the software architecture itself must be optimized for machine readability. GABBE enforces paradigms such as **Clean Architecture** and **Domain-Driven Design (DDD)**. These patterns demand that dependencies only point inward and that features are vertically sliced by domain context.

This creates a “Semantic Firewall.” When a specialized agent (e.g., **eng-backend**) is assigned to modify “Invoicing,” it only loads the `/Domain/Invoicing/` context into its Working Memory. It can safely ignore the rest of the application, thereby drastically reducing the hallucination risk and protecting the “Cost Asymmetry” parameter of the Lethal Trifecta.

GABBE achieves zero-to-one autonomy by structurally encoding decades of established human software engineering and product management best practices directly into its immutable procedural memory (**skills**). The system programmatically forces the swarm to adhere to:

- * **Design Principles:** Strict adherence to SOLID, DRY, and KISS principles is validated during the *Reflect* and *Verify* stages of the RARV loop.
- * **Design Patterns:** Agents utilize established Gang of Four (GoF) patterns (e.g., Strategy, Factory, Repository, CQRS) rather than creating ad-hoc, hallucinated structures.
- * **Quality Assurance:** Test-Driven Development (TDD) and Behavior-Driven Development (BDD) are enforced at the architectural level, preventing code generation until a failing test asserts the requirement.
- * **Product Rigor:** Product personas (**prod-pm**, **prod-architect**) enforce Agile and Lean methodologies to scope deliverables strictly to the validated PRD, preventing feature creep.
- * **Operational Excellence:** CI/CD pipeline generation, Infrastructure-as-Code (IaC), and strict RBAC security validations, handled by the Operations Swarm before deployment.

By binding probabilistic LLMs to recognized deterministic engineering principles, GABBE ensures that the generated output scales identically to enterprise-grade code authored by senior human architects.

VI. COMPARATIVE ANALYSIS AND INDUSTRY IMPLICATIONS

To position GABBE within the existing agentic AI ecosystem, an analysis against leading frameworks highlights its unique agent-agnostic approach.

Tool	What it does	What it lacks vs GABBE
GitHub Copilot	Code completion & chat	No SDLC governance, no memory, no structured workflows
Cursor	AI-native editor with rules	Rules are static, no Active Inference, no multi-agent swarm
Devin	Autonomous agent	Black box, no human-in-the-loop gates, proprietary lock-in
OpenDevin	Open Devin alternative	General purpose, no specialized engineering methodology
Aider	CLI-based coding assistant	Git-aware but no SDLC, no neuro-cognitive orchestration
SWE-agent	Research-grade agent	Academic tool, not production-ready engineering kit
GABBE	Universal governance kit	Works with ANY agent, adds cognition + memory + governance

Table IV. GABBE positioned against existing multi-agent software engineering frameworks.

When benchmarked down to the architectural specifics against 2025/2026 industry standards such as CrewAI and Microsoft Agent Framework (MAF), GABBE introduces critical structural differentiators:

Table V. Architectural Comparables

Feature	GABBE Architecture (2026)	CrewAI	LangGraph
Cognitive Model	System 1 & 2 (Active Inference + Swarm)	Role-Based Prompting	DAG Workflow
Memory Strategy	EPS 4-Layer (Working, Episodic, Semantic, Procedural)	Volatile / Vector DB	Checkpoints

Feature	GABBE Architecture (2026)	CrewAI	LangGraph
Cost Arbitration	Dynamic Local/Remote Routing	External plugin required	Hand-coded
Self-Evolution	Yes (EPO / <code>meta-optimize.skill</code>)	Manual Prompt Ops	None
Governance	Strict SDLC & EARS Spec Enforcement	Non-enforced	Hand-coded logic

GABBE proves that zero-to-one Agentic AI architectures must integrate organizational psychology and neuroscience paradigms. A flat taxonomy of agents fails to scale beyond a trivial threshold of complexity without a meta-cognitive supervisor. The utilization of the Free Energy Principle to drive autonomous project management shifts Agentic AI from “instructed coding tools” to “autonomous systems engineers.”

VII. CONCLUSION

The transition to industrial Agentic Software Engineering demands rigorous scaffolding to ensure determinism, economic viability, and semantic contextualization. The Generative Architectural Brain Base Engine (GABBE) provides this architecture by combining Swarm Intelligence with Active Inference. Through its meta-cognitive Brain Mode, structured 4-layer memory, and cost-effective LLM routing, GABBE directly mitigates “Another Lethal Trifecta” of agentic coding. Its theoretical underpinnings present a universally applicable blueprint for future autonomous systems, bridging the gap between localized stochastic tasks and reliable, enterprise-grade software synthesis.

REFERENCES

- [1] S. Feng, Z. Wang, Y. Wang, S. Ebrahimi, H. Palangi, L. Miculicich, A. Kulshrestha, N. Rauschmayr, Y. Choi, Y. Tsvetkov, C.-Y. Lee, and T. Pfister, “Model Swarms: Collaborative Search to Adapt LLM Experts via Swarm Intelligence,” *arXiv preprint arXiv:2410.11163*, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.11163>
- [2] V. Tawosi, K. Ramani, S. Alamir, and X. Liu, “ALMAS: an Autonomous LLM-based Multi-Agent Software Engineering Framework,” *arXiv preprint arXiv:2510.03463*, Oct. 2025. [Online]. Available: <https://arxiv.org/abs/2510.03463>
- [3] K. Friston, “The free-energy principle: a rough guide to the brain?,” *Trends in Cognitive Sciences*, vol. 13, no. 7, pp. 293-301, Jul. 2009. doi: 10.1016/j.tics.2009.04.005.
- [4] B. Wen, “The Missing Reward: Active Inference in the Era of Experience,” *arXiv preprint arXiv:2508.05619*, Aug. 2025. [Online]. Available: <https://arxiv.org/abs/2508.05619>
- [5] A. N. Besleaga, “GABBE - Generative Architectural Brain Base Engine,” *GitHub repository*. [Online]. Available: <https://github.com/andreibesleaga/GABBE>
- [6] A. N. Besleaga, “GABBE: The Cognitive Engineering Platform,” *Substack*. [Online]. Available: <https://andreibesleaga.substack.com/p/gabbe-the-cognitive-engineering-platform>
- [7] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to Route LLMs with Preference Data,” in *Proc. International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.18665. [Online]. Available: <https://arxiv.org/abs/2406.18665>
- [8] D. B. Acharya, K. Kuppan, and B. Divya, “Agentic AI: Autonomous Intelligence for Complex Goals – A Comprehensive Survey,” *IEEE Access*, Jan. 2025.

[9] Y. Hu, Y. Cai, Y. Du, X. Zhu, X. Liu, Z. Yu, Y. Hou, S. Tang, and S. Chen, “Self-Evolving Multi-Agent Collaboration Networks for Software Development (EvoMAC),” in *Proc. International Conference on Learning Representations (ICLR)*, 2025.